OBJECT ORIENTED PROGRAMMING

Assignment 03

1 Question • Total Marks: 20 • Spring 2026

Course Instructor : Ma'am Hina Iqbal

📧: Hina Iqbal

Teacher Assistant (TA) : Syed Saad Ali

📧: l242549@lhr.nu.edu.pk

Section : BSSE-A, BSSE-B

Question 1

CampusCore

20 Marks

Coding Standards & Assignment Guidelines

Naming Conventions

- Use lowerCamelCase for all variables and functions — e.g., `userDetails`, `calculateTotal`
- Use ALL_CAPS for constants — e.g., `PI`, `MAX_SIZE`

Documentation & Comments

- Add comments wherever the logic is non-trivial or needs explanation
- Every function definition must include a documentation comment block — in Visual Studio, place your cursor above the function and press **Ctrl + /** to auto-generate it

Exception Handling

- Handle invalid user input — assume the user can and will enter anything
- Check for null pointers before dereferencing manually managed memory
- Free or delete every allocation that is no longer in use — no memory leaks
- Your program must remain stable under all conditions — no unexpected crashes or undefined behavior

Academic Integrity

- Plagiarism of any kind will result in a **DC referral** — no exceptions
- You may use tools like ChatGPT to *understand* a concept, never to copy a solution

Time Management

- Do not start two days before the deadline or on weekends — this assignment is long and logic heavy
- Read and understand the problem fully first
- Identify patterns, design your logic on paper, then implement
- If anything about a question is unclear, ask in the **WhatsApp group** beforehand — no excuses will be accepted during evaluation

Regarding use of built-in functions and Pointer-based allocation

The use of any built-in functions is strictly prohibited. Arrays must be accessed using pointer-based allocation *() ; using square brackets [] will result in mark deduction

Question 01 — CampusCore

Aggregation & Composition

Marks: 20 | Topic: Aggregation & Composition

Question 01 — CampusCore

Aggregation & Composition • OOP Relationships • Marks: 20

Problem Overview

You are required to build **CampusCore**, a console-based university simulation in C++ that demonstrates both **Composition** and **Aggregation** relationships across multiple interconnected classes.

📌 **Core Definitions — Read Carefully**

- Composition: The child object's lifetime is FULLY controlled by the parent. If the parent is destroyed, the child is destroyed too. The child has NO meaning outside the parent.
- Aggregation: The child object exists INDEPENDENTLY. The parent only holds a reference/pointer to it. Destroying the parent does NOT destroy the child.
- Rule of thumb: Ask yourself — 'Can this object exist without its parent?' If NO = Composition. If YES = Aggregation.

Relationship Map — Study This First

Relationship	Parent → Child	Child Lifetime	Type
`University → Department`	Departments live inside University	Destroyed with University	Composition
`Department → Course`	Courses live inside Department	Destroyed with Department	Composition
`Professor → Address`	Address lives inside Professor	Destroyed with Professor	Composition

`Student → Address`	Address lives inside Student	Destroyed with Student	Composition
`Department → Professor`	Department holds pointer only	Professor survives removal	Aggregation
`Student → Course`	Student holds pointer only	Course survives drop	Aggregation
`University → Student`	University holds pointer only	Student survives University	Aggregation

Class 1 — Address

Role: Composed inside both `Professor` and `Student`. An address has no meaning without the person it belongs to.

```
c++
```

```
class Address {
private:
    char* street;    // dynamically allocated
    char* city;
    char* country;

public:
    Address(const char* street, const char* city, const char* country);
    Address(const Address& other);       // deep copy
    Address& operator=(const Address& other);
    ~Address();
    void display() const;
};
```

Sample Output:

```
c++
```

```
Address: 47-B Sunset Boulevard, Karachi, Pakistan
```

> ⚠ **Composition Note**
> Address is a VALUE MEMBER inside Professor and Student — not a pointer. It is constructed WITH the parent and destroyed WITH the parent. This is what makes it Composition.

Class 2 — Course

Role: Composed inside `Department`. A Course cannot exist without a Department — if the department is gone, its courses cease to exist.

```
c++
```

```
class Course {
private:
   char* courseCode;   // dynamically allocated
   char* courseName;
   int   creditHours;

public:
   Course(const char* code, const char* name, int credits);
   Course(const Course& other);
   Course& operator=(const Course& other);
   ~Course();
   const char* getCode() const;
   void display() const;
};
```

Sample Output:

```
c++
```

```
[Course] CS301 - Object Oriented Programming  (3 Credit Hours)
```

Class 3 — Professor

Role: Exists **independently** of any Department. A Professor is an independent entity that can be assigned to multiple departments and continues to exist even if a department is removed or destroyed.

Composes: `Address` — the address is a value member, born and destroyed with the Professor.

```
c++
```

```
class Professor {
private:
   char*   name;
   char*   employeeId;
   char*   specialization;
   Address address;   // COMPOSITION — value member, not a pointer
```

```
public:
    Professor(const char* name, const char* id,
              const char* specialization, const Address& addr);
    Professor(const Professor& other);
    Professor& operator=(const Professor& other);
    ~Professor();
    const char* getId() const;
    void display() const;
};
```

Sample Output:

```
c++
```

```
========================================
  Professor: Ma'am Anoosha Khan  [ID: P-1001]
  Specialization: Object Oriented Programming
  Address: 47-B Sunset Boulevard, Karachi, Pakistan
========================================
```

Class 4 — Student

Role: Exists **independently** of the University. Students are created outside and registered with the university. If the university is destroyed, students still exist.

Composes: `Address` — value member, destroyed with the Student.

Aggregates: `Course*` pointers — student enrolls in courses owned by departments, NOT by the student. Dropping a course does NOT delete it.

```
c++
```

```
class Student {
private:
    char*    name;
    char*    rollNumber;
    int      semester;
    Address  address;        // COMPOSITION — value member
    Course** enrolledCourses; // AGGREGATION — pointers only
    int      courseCount;

public:
    Student(const char* name, const char* roll,
            int semester, const Address& addr);
    ~Student();   // does NOT delete the courses
```

```
    void enrollCourse(Course* course);
    void dropCourse(const char* courseCode);
    const char* getRoll() const;
    void display() const;
};
```

Function	Behavior	Edge Case
`enrollCourse(Course*)`	Add course pointer to enrolled list. Grow array by 1.	Throw/print error if already enrolled in this course.
`dropCourse(const char*)`	Remove pointer from list. Shift array. Do NOT delete the course object.	Print error if course not found in student's list.
`display()`	Show name, roll, semester, address, and all enrolled courses.	Print 'No courses enrolled' if list is empty.

Sample Output:

```
c++
```

```
========================================
  Student: Ali Hassan  [Roll: 22I-1045]
  Semester: 5
  Address: 7-D Johar Town, Lahore, Pakistan
  Enrolled Courses:
    [1] CS301 - Object Oriented Programming  (3 Cr)
    [2] CS302 - Data Structures               (3 Cr)
    [3] CS401 - Computer Networks             (3 Cr)
========================================
```

Class 5 — Department

Role: Composed inside `University`. Departments are created internally and destroyed when the University is destroyed.

Composes: `Course[]` — course objects live inside the department. When the department dies, its courses are gone.

Aggregates: `Professor*` — department only holds pointers to externally-existing professors. Removing or destroying the department does NOT delete professors.

c++

```
class Department {
private:
    char*       departmentName;
    char*       departmentCode;
    Course*     courses;     // COMPOSITION — value array
    int         courseCount;
    Professor** professors;  // AGGREGATION — pointer array
    int         professorCount;

public:
    Department(const char* name, const char* code);
    ~Department();   // deletes courses, does NOT delete professors

    void   addCourse(const char* code, const char* name, int credits);
    void   assignProfessor(Professor* prof);
    void   removeProfessor(const char* employeeId);
    Course* getCourse(const char* courseCode);
    const char* getCode() const;
    void   display() const;
};
```

Function	Behavior	Edge Case
`addCourse(code, name, credits)`	Create a new Course object inside the department's courses array. Resize by 1.	—
`assignProfessor(Professor*)`	Add professor pointer to array. Resize by 1.	Print error if professor already assigned to this department.
`removeProfessor(employeeId)`	Remove professor pointer from array. Shift. Do NOT delete the Professor object.	Print error if professor not found.
`getCourse(courseCode)`	Return pointer to the matching Course object. Returns nullptr if not found.	—
`display()`	Show department name, code, all courses, all assigned professors.	Print 'No courses' or 'No professors' if lists are empty.

Sample Output:

```
c++
```

```
========================================
  Department: Computer Science  [CS]
----------------------------------------
  Courses Offered:
   [1] CS301 - Object Oriented Programming  (3 Cr)
   [2] CS302 - Data Structures                (3 Cr)
   [3] CS401 - Computer Networks              (3 Cr)

  Assigned Professors:
   [1] Ma'am Anoosha Khan   [P-1001]  | OOP
   [2] Sir Razi Uddin       [P-1002]  | DSA
========================================
```

Class 6 — University

Role: Top-level class. Owns its Departments (Composition). Registers Students externally (Aggregation).

Composes: `Department[]` — departments are created inside and destroyed with the university.

Aggregates: `Student*` — students exist independently. Destroying the university does NOT destroy students.

```
c++
```

```
class University {
private:
   char*        universityName;
   char*        location;
   Department*  departments;   // COMPOSITION — value array
   int          deptCount;
   Student**    students;   // AGGREGATION — pointer array
   int          studentCount;

public:
   University(const char* name, const char* location);
   ~University();   // deletes departments, does NOT delete students

   void        addDepartment(const char* name, const char* code);
   Department* getDepartment(const char* code);
   void        registerStudent(Student* student);
   void        removeStudent(const char* rollNumber);
   void        display() const;
};
```

Function	Behavior	Edge Case
`addDepartment(name, code)`	Create new Department object inside internal array. Resize by 1.	—
`getDepartment(code)`	Return pointer to matching Department. Returns nullptr if not found.	—
`registerStudent(Student*)`	Add student pointer to array. Resize by 1.	Print error if student already registered.
`removeStudent(rollNumber)`	Remove student pointer. Shift array. Do NOT delete the Student object.	Print error if student not found.
`display()`	Print full university breakdown: all departments with courses & professors, all students.	—

Sample Output:

```
c++
```

```
========================================
  UNIVERSITY: FAST-NUCES  |  Lahore, Pakistan
========================================

  [Department 1] Computer Science [CS]
   Courses: CS301, CS302, CS401
   Professors: Ma'am Anoosha Khan, Sir Razi Uddin

  [Department 2] Electrical Engineering [EE]
   Courses: EE201, EE305
   Professors: Ma'am Arooj Khalil

  Registered Students: 3
   [1] Ali Hassan     [22I-1045]  Semester 5
   [2] Sara Ahmed     [22I-1062]  Semester 5
   [3] Hamza Malik    [22I-1078]  Semester 3
========================================
```

What main() Must Demonstrate — Step by Step

Your `main()` must walk through all the steps below in order. Every step must print clearly labeled output so it is obvious what concept is being proven.

▸ Step 1 — Create Professors Independently (proving they exist outside any Department)

📥 Input	📤 Expected Output

`// Professors created in main — NO department yet`
`Professor p1("Ma'am Anoosha Khan", "P-1001", "OOP", addr1);`
`Professor p2("Sir Razi Uddin", "P-1002", "DSA", addr2);`
`Professor p3("Ma'am Arooj Khalil", "P-1003", "Networks", addr3);`
`Professor p4("Sir Zeeshan Ali Rana","P-1004","Databases", addr4);`
`cout << "Professors created independently:";`
`p1.display();  p2.display();`
	`Professors created independently:`
	`Professor: Ma'am Anoosha Khan [P-1001]`
	`  OOP | 47-B Sunset Boulevard, Karachi`
	`Professor: Sir Razi Uddin  [P-1002]`
	`  DSA | 9-C Moonlight Avenue, Peshawar`
	`  (No department assigned yet)`

▸ Step 2 — Build University and Add Departments (Composition)

📥 Input	📤 Expected Output

`University uni("FAST-NUCES", "Lahore, Pakistan");`
 	
`uni.addDepartment("Computer Science", "CS");`
`uni.addDepartment("Electrical Engineering", "EE");`
`cout << "University created with departments:";`
	`University created: FAST-NUCES | Lahore, Pakistan`
	
	`> Department CS added: Computer Science`
	`> Department EE added: Electrical Engineering`
	
	`Note: Departments are COMPOSED inside`
	`University. They cannot exist without it.`

▸ Step 3 — Add Courses to Departments (Composition)

📥 Input	📤 Expected Output

`Department* cs = uni.getDepartment("CS");`
`cs->addCourse("CS301", "Object Oriented Programming", 3);`
`cs->addCourse("CS302", "Data Structures", 3);`
`cs->addCourse("CS401", "Computer Networks", 3);`
 	
`Department* ee = uni.getDepartment("EE");`
`ee->addCourse("EE201", "Circuit Analysis", 3);`
`ee->addCourse("EE305", "Signals & Systems", 3);`
	`Courses added to CS:`
	`  > CS301 - Object Oriented Programming  (3 Cr)`
	`  > CS302 - Data Structures (3 Cr)`
	`  > CS401 - Computer Networks (3 Cr)`
	
	`Courses added to EE:`
	`  > EE201 - Circuit Analysis  (3 Cr)`
	`  > EE305 - Signals & Systems (3 Cr)`
	
	`Note: Courses OWNED by Department.`
	`They cannot exist without it. (Composition)`

▸ Step 4 — Assign Professors to Departments (Aggregation)

📥 Input	📤 Expected Output

`cs->assignProfessor(&p1);  // Anoosha Khan`
`cs->assignProfessor(&p2);  // Razi Uddin`
`ee->assignProfessor(&p3);  // Arooj Khalil`
`ee->assignProfessor(&p4);  // Zeeshan Ali Rana`
 	
`// Same professor in two departments:`
`cs->assignProfessor(&p3);  // Arooj Khalil also in CS`
	`> Ma'am Anoosha Khan  assigned to CS`
	`> Sir Razi Uddin  assigned to CS`
	`> Ma'am Arooj Khalil  assigned to EE`
	`> Sir Zeeshan Ali Rana assigned to EE`
	`> Ma'am Arooj Khalil  assigned to CS`
	
	`Note: Professors NOT owned by Department.`
	`Same prof can be in multiple depts. (Aggregation)`

▸ Step 5 — Duplicate Assignment (Edge Case)

📥 Input	📤 Expected Output
`// Try to assign Anoosha Khan to CS again:` `cs->assignProfessor(&p1);`	`Error: Ma'am Anoosha Khan [P-1001] is already` `assigned to Department CS.`

▸ Step 6 — Create Students and Enroll in Courses (Aggregation)

📥 Input	📤 Expected Output

`Address sAddr("7-D Johar Town", "Lahore", "Pakistan");`
`Student s1("Ali Hassan", "22I-1045", 5, sAddr);`
 	
`Course* oop  = cs->getCourse("CS301");`
`Course* ds   = cs->getCourse("CS302");`
`Course* nets = cs->getCourse("CS401");`
 	
`s1.enrollCourse(oop);`
`s1.enrollCourse(ds);`
`s1.enrollCourse(nets);`
`s1.display();`
	`Student created: Ali Hassan [22I-1045]`
	
	`> Enrolled in CS301 - Object Oriented Programming`
	`> Enrolled in CS302 - Data Structures`
	`> Enrolled in CS401 - Computer Networks`
	
	`Student display:`
	`  Ali Hassan | Roll: 22I-1045 | Sem: 5`
	`  Address: 7-D Johar Town, Lahore`
	`  Enrolled: CS301, CS302, CS401`
	
	`Note: Student does NOT own these courses.`
	`(Aggregation)`

▸ Step 7 — Duplicate Enrollment (Edge Case)

📥 Input	📤 Expected Output
`// Ali tries to enroll in CS301 again:` `s1.enrollCourse(oop);`	`Error: Ali Hassan is already enrolled in` `CS301 - Object Oriented Programming.`

▸ Step 8 — Drop a Course (Aggregation Proof)

📥 Input	📤 Expected Output

`s1.dropCourse("CS401");`
 	
`// Now check if CS401 still exists:`
`Course* check =`
`cs->getCourse("CS401");`
`if (check) check->display();`
	`> CS401 removed from Ali Hassan's enrolled list.`
	
	`CS401 still exists in CS Department:`
	`  [Course] CS401 - Computer Networks (3 Cr)`
	
	`Course was NOT deleted — Aggregation confirmed.`
	`Ali dropped it; the course still belongs`
	`to the CS Department.`

▸ Step 9 — Remove a Professor from Department (Aggregation Proof)

📥 Input	📤 Expected Output

`cs->removeProfessor("P-1002");  // remove Razi Uddin`
 	
`// Confirm professor still exists:`
`p2.display();`
	`> Sir Razi Uddin [P-1002] removed from CS.`
	
	`Sir Razi Uddin still exists independently:`
	`  Professor: Sir Razi Uddin [P-1002]`
	`  Specialization: DSA`
	`  Address: 9-C Moonlight Avenue, Peshawar`
	
	`Professor was NOT deleted — Aggregation confirmed.`

▸ Step 10 — Register Students with University (Aggregation)

📥 Input	📤 Expected Output

`uni.registerStudent(&s1);`
`uni.registerStudent(&s2);`
`uni.registerStudent(&s3);`
 	
`// Try registering s1 again:`
`uni.registerStudent(&s1);`
	`> Ali Hassan  [22I-1045] registered.`
	`> Sara Ahmed  [22I-1062] registered.`
	`> Hamza Malik [22I-1078] registered.`
	
	`Error: Ali Hassan [22I-1045] is already`
	`registered in the university.`

▸ Step 11 — Destroy University (Composition + Aggregation Proof)

📥 Input	📤 Expected Output

`// University goes out of scope or is deleted:`
`{`
`  University tempUni("COMSATS", "Islamabad");`
`  tempUni.addDepartment("CS", "CS");`
`  Department* d = tempUni.getDepartment("CS");`
`  d->addCourse("CS101", "Intro to CS", 3);`
`  d->assignProfessor(&p4);`
`  tempUni.registerStudent(&s1);`
`} // tempUni destroyed here`
 	
`// Confirm p4 and s1 still exist:`
`p4.display();`
`s1.display();`
	`tempUni is being destroyed...`
	`  > Department CS destroyed.`
	`  Courses gone: CS101 (Composition)`
	
	`But externally created objects survive:`
	
	`Sir Zeeshan Ali Rana [P-1004] — still alive`
	`  (Aggregation confirmed — not owned by dept)`
	
	`Ali Hassan [22I-1045] — still alive`
	`  (Aggregation confirmed — not owned by uni)`

▸ Step 12 — Destroy Professor (Composition Proof — Address Gone With Them)

📥 Input	📤 Expected Output

`// Professor goes out of scope:`
`{`
`  Address tmpAddr("99-Z Test Street",`
`  "Multan", "Pakistan");`
`  Professor tmpProf("Dr. Test", "P-9999",`
`  "Testing", tmpAddr);`
`  tmpProf.display();`
`} // tmpProf destroyed here — address too`
 	
`cout << "Professor destroyed.";`
	`Professor: Dr. Test  [P-9999]`
	`  Specialization: Testing`
	`  Address: 99-Z Test Street, Multan, Pakistan`
	
	`Professor destroyed.`
	`  > Address (99-Z Test Street, Multan)`
	`  destroyed WITH the professor.`
	
	`Address was NOT independent — Composition confirmed.`

Edge Cases — All Must Be Handled

Scenario	Expected Behavior
Enroll student in a course they are already enrolled in	Print error: 'Already enrolled in [courseCode]'
Assign professor to a department they are already in	Print error: 'Already assigned to [deptCode]'
Drop a course the student is not enrolled in	Print error: 'Course [code] not found in enrolled list'
Remove a professor not in the department	Print error: 'Professor [id] not found in department'
Register the same student twice	Print error: 'Student [roll] already registered'
getDepartment() with non-existent code	Return nullptr, caller must check and handle
getCourse() with non-existent code	Return nullptr, caller must check and handle

`Enroll in a course after its department is destroyed`	Course pointer is dangling — undefined behavior to avoid. Demo in step 11.

📋 Submission Format

- File name: YourRollNumber_A3_Q1.cpp
- Add a comment block at top: Name | Roll Number | Section | Date
- Late submissions: -5 marks per day unless prior approval granted.
- If anything about the question (not the logic) is unclear (figure it out yourself pls)

Good Luck! (hopefully)